

Azure AI Search Policy-Document Chunking Implementation Guide

Policy Assistant RAG using Azure Blob Storage, Azure AI Search, Microsoft Foundry embeddings, and an LLM

Primary implementation point: configure chunking in the Azure AI Search skillset's Text Split skill. Hybrid search is implemented in the application query, and semantic ranking is configured on the search index.

1. Current Architecture and Target

The current Policy Assistant RAG solution includes Azure AI Search, an Azure AI Search vector index, a Microsoft Foundry text-embedding-3-small deployment, a Microsoft Foundry chat model, Azure Blob Storage, and an ingestion pipeline from Blob Storage into Azure AI Search.

The desired initial configuration is:

Setting	Recommendation
Chunk size	800-1,200 tokens
Overlap	10-15%
Initial document count	5-10 files
Search type	Hybrid search, where available
Semantic ranker	Enable and evaluate
Metadata	Document name, policy category, effective date, source path

2. Where to Make the Chunking Change

Make the change in Azure AI Search, inside the skillset associated with the Blob Storage indexer. The relevant component is the Text Split skill.

The processing sequence is:

1. Azure Blob Storage provides the source documents.
2. The Azure AI Search indexer reads each document.
3. The skillset's Text Split skill creates text chunks.
4. The Azure OpenAI Embedding skill generates an embedding for each chunk.
5. Index projections write one child search document per chunk into the Azure AI Search index.

Do not configure chunk size in the LLM deployment, the embedding deployment, Blob Storage, or the vector field itself.

3. Recommended Starting Configuration

To implement the screenshot's strategy, use a midpoint configuration of approximately 1,000 tokens with 125 tokens of overlap. This is a 12.5% overlap.

```
"maximumPageLength": 1000,  
"pageOverlapLength": 125,  
"unit": "azureOpenAITokens"
```

For evaluation, retain a second baseline using smaller chunks, such as 512 tokens with 128 tokens of overlap. Policy documents sometimes benefit from larger chunks because rules, exceptions, definitions, and approval steps may span multiple paragraphs; however, the correct setting should be selected through retrieval testing.

4. Portal Implementation Steps

6. **Open the search service:** In the Azure portal, open the Azure AI Search service used by the Policy Assistant.
7. **Open the skillset:** Select Skillsets, then open the skillset created by the Import and vectorize data workflow.
8. **Find the Text Split skill:** In the JSON definition, locate the object whose @odata.type is #Microsoft.Skills.Text.SplitSkill.
9. **Change the split settings:** Set textSplitMode to pages, maximumPageLength to 1000, pageOverlapLength to 125, and unit to azureOpenAITokens.
10. **Preserve downstream paths:** Keep the output target name, commonly pages, unless every downstream skill and index projection is updated.
11. **Save and test:** Use a debug session with one representative policy document before rebuilding the entire index.

5. Text Split Skill Example

```
{  
  "@odata.type": "#Microsoft.Skills.Text.SplitSkill",  
  "name": "#1",  
  "description": "Split policy documents into approximately 1000-token chunks",  
  "context": "/document",  
  "textSplitMode": "pages",  
  "maximumPageLength": 1000,  
  "pageOverlapLength": 125,  
  "maximumPagesToTake": 0,  
  "unit": "azureOpenAITokens",  
  "azureOpenAITokenizerParameters": {  
    "encoderModelName": "cl100k_base"  
  },  
  "defaultLanguageCode": "en",  
  "inputs": [  
    {  
      "name": "text",  
      "source": "/document/content"
```

```

    }
  ],
  "outputs": [
    {
      "name": "textItems",
      "targetName": "pages"
    }
  ]
}

```

Your existing skillset may use different names or paths. Modify only the applicable properties rather than replacing the entire skillset with the sample.

6. Verify the Embedding Skill

The embedding skill should run once for every chunk. Its context and input normally point to `/document/pages/*`.

```

{
  "@odata.type": "#Microsoft.Skills.Text.AzureOpenAIEmbeddingSkill",
  "name": "#2",
  "context": "/document/pages/*",
  "deploymentName": "text-embedding-3-small",
  "modelName": "text-embedding-3-small",
  "dimensions": 1536,
  "inputs": [
    {
      "name": "text",
      "source": "/document/pages/*"
    }
  ],
  "outputs": [
    {
      "name": "embedding",
      "targetName": "vector"
    }
  ]
}

```

The embedding skill's dimensions must match the dimensions configured on the vector field in the search index. Do not change dimensions solely to implement chunking.

7. Verify Index Projections

Index projections should create one search document per chunk and copy the parent identifier and metadata to each child chunk.

```

"indexProjections": {
  "selectors": [
    {
      "targetIndexName": "policy-index-v2",

```

```

"parentKeyFieldName": "parent_id",
"sourceContext": "/document/pages/*",
"mappings": [
  {
    "name": "chunk",
    "source": "/document/pages/*"
  },
  {
    "name": "vector",
    "source": "/document/pages/*/vector"
  },
  {
    "name": "document_name",
    "source": "/document/metadata_storage_name"
  },
  {
    "name": "source_path",
    "source": "/document/metadata_storage_path"
  }
]
},
"parameters": {
  "projectionMode": "skipIndexingParentDocuments"
}
}

```

8. Add Recommended Metadata

Each indexed chunk should contain enough metadata to identify, filter, cite, and trace the source policy.

Field	Type	Suggested attributes
document_name	Edm.String	Searchable, filterable, retrievable
policy_category	Edm.String	Searchable, filterable, facetable, retrievable
effective_date	Edm.DateTimeOffset	Filterable, sortable, retrievable
source_path	Edm.String	Filterable, retrievable

Azure Blob indexers expose built-in values such as `metadata_storage_name` and `metadata_storage_path`. Policy category and official effective date are business metadata and normally need to be supplied as custom blob metadata, extracted by a custom skill, or maintained through another authoritative source.

9. Test with a Debug Session

- Select one representative policy containing headings, definitions, requirements, exceptions, and approval steps.
- Run the skillset in a debug session.
- Inspect `/document/pages` and confirm that multiple chunks were produced.
- Confirm each chunk has text, an embedding vector, a parent ID, and the required metadata.

- Review chunk boundaries to ensure important sentences or numbered requirements are not divided poorly.

10. Reindex the Documents

After saving the skillset, reset and run the associated indexer so existing source documents are processed again.

12. Open Indexers in the Azure AI Search service.
13. Select the policy indexer.
14. Choose Reset.
15. Choose Run.
16. Review execution history for warnings and failures.

For a safer proof of concept, create a new index such as policy-index-v2 and point a copied skillset and indexer to it. This prevents old or orphaned chunks from affecting the comparison.

11. Implement Hybrid Search

Hybrid search is a query-time capability. Your Azure Function or application should send both a keyword query and a vector query. Azure AI Search combines the result lists.

```
{
  "search": "How many days of remote work are employees allowed?",
  "vectorQueries": [
    {
      "kind": "text",
      "text": "How many days of remote work are employees allowed?",
      "fields": "vector",
      "k": 50
    }
  ],
  "select": "chunk,document_name,policy_category,effective_date,source_path",
  "top": 10
}
```

12. Configure Semantic Ranking

Semantic ranking is configured on the search index and requested at query time. A practical semantic configuration is:

- Title field: document_name
- Content field: chunk
- Keyword field: policy_category

```
{
  "search": "What is the approval process for working remotely?",
  "queryType": "semantic",
  "semanticConfiguration": "policy-semantic-config",

```

```

"vectorQueries": [
  {
    "kind": "text",
    "text": "What is the approval process for working remotely?",
    "fields": "vector",
    "k": 50
  }
],
"select": "chunk,document_name,policy_category,effective_date,source_path",
"top": 10
}

```

13. Evaluation Plan

Use 20-30 representative policy questions and compare at least two chunking configurations.

Test	Chunk size	Overlap	Purpose
A	512 tokens	128 tokens	Smaller-chunk baseline
B	1,000 tokens	125 tokens	Screenshot strategy

- Was the correct policy retrieved?
- Was the exact supporting passage in the top results?
- Were exceptions and qualifying language preserved?
- Did the answer cite the correct document and source path?
- Did retrieval return excessive unrelated text?

14. Recommended Implementation Sequence

17. Keep the current index as a baseline.
18. Create policy-index-v2 for the new chunking strategy.
19. Update the Text Split skill to 1,000 tokens and 125-token overlap.
20. Verify the embedding skill and vector dimensions.
21. Copy metadata into every projected child chunk.
22. Run a debug session against one policy document.
23. Reset and run the test indexer.
24. Implement hybrid search in the Azure Function or application.
25. Add a semantic configuration to the index.
26. Evaluate the two chunking configurations with representative policy questions.

15. Key Decision Summary

Capability	Where it is configured
Chunking	Azure AI Search skillset - Text Split skill
Embedding generation	Azure OpenAI Embedding skill in the skillset
Chunk-to-index mapping	Skillset index projections
Vector field	Azure AI Search index
Hybrid retrieval	Application or Azure Function query
Semantic ranking	Index semantic configuration and query

	parameters
Source metadata	Blob metadata/extraction plus index projection mappings

References

- [Microsoft Learn - Integrated vectorization in Azure AI Search](#)
- [Microsoft Learn - Text Split cognitive skill](#)
- [Microsoft Learn - Azure OpenAI Embedding skill](#)
- [Microsoft Learn - Index projections in Azure AI Search](#)
- [Microsoft Learn - Debug sessions for skillsets](#)
- [Microsoft Learn - Reset and run indexers](#)
- [Microsoft Learn - Hybrid search overview and query guidance](#)
- [Microsoft Learn - Configure semantic ranking](#)